

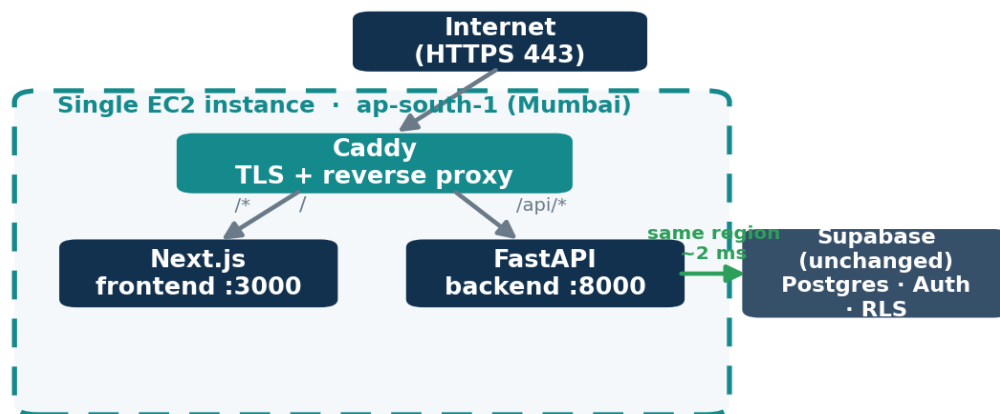
Finance Operations

AWS Deployment Runbook — for the deploying engineer

24 July 2026 • UpsureAI • Stack: FastAPI + Next.js 15 + Supabase (unchanged)

0. What you are deploying

Two containers on a single EC2 instance, behind Caddy (auto-HTTPS reverse proxy). The database, authentication (Google OAuth) and row-level security all stay on **Supabase** — **do not migrate or recreate them**. The app already connects to Supabase via the **transaction pooler** (host `...pooler.supabase.com:6543`), which is correct — leave it as is. Migrations use the **direct** connection (port 5432) and run automatically on backend start.



Caddy routes `/api/*` to the FastAPI backend (`:8000`) and everything else to the Next.js frontend (`:3000`). Because both are served from one domain, this is same-origin — CORS is effectively a non-issue.

1. Provision the instance

- **Type:** EC2 **t3.large** (2 vCPU / 8 GB). Enable **T3 Unlimited** to avoid CPU-credit throttling under sustained load.
- **OS:** Ubuntu Server 24.04 LTS (x86_64).
- **Region:** **ap-south-1 (Mumbai)** — same region as Supabase; keeps DB latency ~2 ms.
- **Storage:** 30 GB gp3 EBS (root).
- **Networking:** allocate a static **Elastic IP** and associate it (the DNS A-record will point here).

2. Security group (firewall)

Inbound 443	TCP from 0.0.0.0/0 - public HTTPS
Inbound 80	TCP from 0.0.0.0/0 - HTTP, used only for the automatic certificate + redirect to 443
SSH (22)	DO NOT open to the internet - use AWS SSM Session Manager for shell access
Outbound	Allow all (needed for Supabase, Groq, Google, SMTP, package installs)

Attach an IAM role with `AmazonSSMManagedInstanceCore` so you can open a shell via Session Manager without exposing port 22.

3. Install Docker + Compose

```
# On the instance (Ubuntu 24.04), via SSM shell:
sudo apt-get update && sudo apt-get install -y ca-certificates curl git
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo tee /etc/apt/keyrings/docker.asc > /dev/null
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \
  https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo $VERSION_CODENAME) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
sudo usermod -aG docker $USER # re-open the shell afterwards
```

4. Secrets (do NOT commit these)

Store all secrets in **AWS SSM Parameter Store** (SecureString) and render them into a local `secrets/backend.env` at deploy time. Keep this file out of git. The full variable list is in the reference tables at the end. Backend secrets load via `env_file`; the frontend's `NEXT_PUBLIC_*` values are **build-time** arguments (see step 6).

Security follow-ups (from the audit) to apply during this move: confirm **APP_ENV=production** on the instance (a dev value bypasses authentication); set real high-entropy **SECRET_KEY** and **GOOGLE_WEBHOOK_SECRET**; and keep every key in SSM, never in a committed file. Rotating the Supabase/Groq keys is a dashboard action on your side.

5. Deployment artefacts

Three files are added to the repo root / frontend. The **backend Dockerfile already exists** and needs no change.

5a. One-line change to frontend/next.config.ts

Add output: 'standalone' so the frontend produces a small, self-contained server image:

```
const nextConfig: NextConfig = {
  output: 'standalone', // <-- add this line
  eslint: { ignoreDuringBuilds: true },
  typescript: { ignoreBuildErrors: true },
  images: { unoptimized: true },
  // ...existing headers() stay unchanged
};
```

5b. frontend/Dockerfile (new)

```
# ---- build stage ----
FROM node:20-slim AS builder
WORKDIR /app
# NEXT_PUBLIC_* are compiled INTO the bundle, so they must exist at build time:
ARG NEXT_PUBLIC_API_URL
ARG NEXT_PUBLIC_APP_URL
ARG NEXT_PUBLIC_SUPABASE_URL
ARG NEXT_PUBLIC_SUPABASE_ANON_KEY
ARG NEXT_PUBLIC_VAPID_PUBLIC_KEY
ARG NEXT_PUBLIC_ADMIN_EMAIL
ENV NEXT_PUBLIC_API_URL=$NEXT_PUBLIC_API_URL \
    NEXT_PUBLIC_APP_URL=$NEXT_PUBLIC_APP_URL \
    NEXT_PUBLIC_SUPABASE_URL=$NEXT_PUBLIC_SUPABASE_URL \
    NEXT_PUBLIC_SUPABASE_ANON_KEY=$NEXT_PUBLIC_SUPABASE_ANON_KEY \
    NEXT_PUBLIC_VAPID_PUBLIC_KEY=$NEXT_PUBLIC_VAPID_PUBLIC_KEY \
    NEXT_PUBLIC_ADMIN_EMAIL=$NEXT_PUBLIC_ADMIN_EMAIL \
    NEXT_TELEMETRY_DISABLED=1
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# ---- runtime stage ----
FROM node:20-slim AS runner
WORKDIR /app
ENV NODE_ENV=production NEXT_TELEMETRY_DISABLED=1
RUN useradd -m -u 1001 nextjs
COPY --from=builder /app/public ./public
COPY --from=builder --chown=nextjs:nextjs /app/.next/standalone ./
COPY --from=builder --chown=nextjs:nextjs /app/.next/static ./next/static
USER nextjs
EXPOSE 3000
CMD ["node", "server.js"]
```

5c. docker-compose.prod.yml (new, repo root)

Note: this is a **separate** file from the existing backend/docker-compose.yml, which is dev-only (it spins up a throwaway local Postgres) — **do not use the dev file in production**.

```
services:
  backend:
    build: { context: ./backend, dockerfile: Dockerfile }
    env_file: ./secrets/backend.env          # DATABASE_URL(pooler), MIGRATION_DATABASE_URL(direct), keys
    command: >
      sh -c "alembic upgrade head &&
      uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 2"
    expose: ["8000"]
    restart: unless-stopped

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    args:
      NEXT_PUBLIC_API_URL: https://finance.yourcompany.com/api/v1
      NEXT_PUBLIC_APP_URL: https://finance.yourcompany.com
      NEXT_PUBLIC_SUPABASE_URL: ${NEXT_PUBLIC_SUPABASE_URL}
      NEXT_PUBLIC_SUPABASE_ANON_KEY: ${NEXT_PUBLIC_SUPABASE_ANON_KEY}
      NEXT_PUBLIC_VAPID_PUBLIC_KEY: ${NEXT_PUBLIC_VAPID_PUBLIC_KEY}
      NEXT_PUBLIC_ADMIN_EMAIL: ${NEXT_PUBLIC_ADMIN_EMAIL}
    expose: ["3000"]
    restart: unless-stopped

  caddy:
    image: caddy:2
    ports: ["80:80", "443:443"]
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy_data:/data
      - caddy_config:/config
    depends_on: [backend, frontend]
    restart: unless-stopped

volumes:
  caddy_data:
  caddy_config:
```

5d. Caddyfile (new, repo root)

```
finance.yourcompany.com {
  encode gzip
  # API traffic -> FastAPI (routes live under /api/v1)
  handle /api/* {
    reverse_proxy backend:8000
  }
  # Backend health check (optional, for uptime monitoring)
  handle /health {
    reverse_proxy backend:8000
  }
  # Everything else -> Next.js
  handle {
    reverse_proxy frontend:3000
  }
}
```

Replace `finance.yourcompany.com` with the real domain in both the Caddyfile and the frontend build args. Caddy obtains and renews the TLS certificate automatically (Let's Encrypt) — no manual cert steps.

6. Build & deploy

Build discipline: the `NEXT_PUBLIC_*` values are baked into the frontend at build time, so a rebuild is required whenever the domain or those values change. The Next.js build peaks near ~3 GB RAM; on `t3.large` (8 GB) it is safe. For the very first deploy there is no live traffic, so building on the box is fine. For later updates, prefer building the image separately (or in CI) and pulling it, so a rebuild never competes with live users.

```
# On the instance, from the repo root:
git clone <repo-url> app && cd app          # first time
mkdir -p secrets && <render SSM values into secrets/backend.env>
docker compose -f docker-compose.prod.yml up -d --build
docker compose -f docker-compose.prod.yml ps
docker compose -f docker-compose.prod.yml logs -f backend  # watch migrations, then uvicorn
```

7. DNS

- Create an **A record** for the chosen domain/subdomain pointing to the instance's **Elastic IP**.
- If the domain is in Route 53, do it there; if it is the client's existing domain (Option 2), their IT adds the one record.
- Wait for propagation, then Caddy will issue the certificate automatically on first HTTPS request.

8. Post-deploy: Supabase & Google (critical)

Sign-in will break until these point at the new domain:

- **Supabase** → **Authentication** → **URL Configuration**: set **Site URL** and add the domain to **Redirect URLs**.
- **Google Cloud Console** → **OAuth client**: add the domain to **Authorised JavaScript origins** and **redirect URIs**.
- **Backend** `CORS_ORIGINS`: set to include the new app URL (belt-and-braces; same-origin means it is rarely exercised).

9. Smoke test

- `https://DOMAIN/health` returns 200.
- Load `https://DOMAIN/login`; sign in with Google as a super-admin.
- Create a test deposit request via the public form; confirm it saves and appears in the dashboard.
- Verify a merchandiser sees only their own rows (role scoping intact).

10. Resilience & ops

- **Backups**: schedule daily EBS snapshots via **Data Lifecycle Manager** (e.g. 7-day retention).
- **Auto-recovery**: add a CloudWatch alarm on `StatusCheckFailed_System` with an EC2 recover action.
- **Golden image**: after a known-good deploy, take an **AMI** for fast rebuild.
- **Updates**: `git pull && docker compose -f docker-compose.prod.yml up -d --build` (frontend rebuilds; backend runs migrations on start).
- **Logs**: `docker compose -f docker-compose.prod.yml logs -f [service]`.

Appendix A — Backend environment variables

APP_ENV	MUST be 'production' (any other value enables a dev auth bypass)
APP_DEBUG	false
SECRET_KEY	required - set a real high-entropy value
SUPABASE_URL	https://<ref>.supabase.co
SUPABASE_ANON_KEY	Supabase anon key
SUPABASE_SERVICE_ROLE_KEY	Supabase service role key (keep in SSM only)
SUPABASE_JWT_SECRET	Supabase JWT secret (token verification)
DATABASE_URL	POOLER url - ...pooler.supabase.com:6543 ...cache_size=0 (app traffic)
MIGRATION_DATABASE_URL	DIRECT url - db.<ref>.supabase.co:5432 (alembic migrations)
CORS_ORIGINS	JSON list incl. the app URL, e.g. ["https://finance.yourcompany.com"]
GOOGLE_SERVICE_ACCOUNT_JSON	Drive service-account JSON (TT copies)
GOOGLE_DRIVE_FOLDER_ID	Shared-drive folder id
GOOGLE_WEBHOOK_SECRET	set a real value (currently a placeholder)
VAPID_PRIVATE_KEY	web-push keys (empty => push disabled)
SMTP_HOST / PORT	email (empty host => email disabled)

Appendix B — Frontend build-time variables (NEXT_PUBLIC_*)

NEXT_PUBLIC_API_URL	https://DOMAIN/api/v1 (baseUrl for API calls)
NEXT_PUBLIC_APP_URL	https://DOMAIN
NEXT_PUBLIC_SUPABASE_URL	https://<ref>.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY	Supabase anon key (public by design)
NEXT_PUBLIC_VAPID_PUBLIC_KEY	web-push public key
NEXT_PUBLIC_ADMIN_CONTACT	admin contact shown in-app

Figures for cost/latency/memory are planning estimates for ap-south-1, not billed quotes. Supabase remains the system of record for data, auth and RLS throughout.